

TP -- Scrapy AlloCine & Boursorama

Framework industriel : spiders, items, pipelines, settings.

Deux projets Scrapy fonctionnels en une journée.

5 heures de developpement

Objectifs pedagogiques

A l'issue de ce TP vous serez capable de :

- Créer un projet Scrapy complet (startproject, genspider)
- Écrire un spider avec follow des liens (liste -> fiche detail)
- Définir des Items et des Pipelines (CleanPipeline, SQLitePipeline)
- Configurer settings.py : ROBOTSTXT_OBEY, throttling, retry
- Utiliser scrapy shell pour valider un selecteur en 30 secondes
- Exporter en JSON et CSV via Feed Exports

LIVRABLES ATTENDUS

allocine/ : projet Scrapy complet (items.py, pipelines.py, settings.py, spider)
films.json : top 200 films AlloCine avec 5 champs
films.csv : meme donnees en CSV
boursorama/ : deuxieme projet Scrapy (ou spider supplementaire)
bourse.db : SQLite avec table actions, UNIQUE(isin)

Setup initial -- 15 minutes

Installez Scrapy et initialisez les deux projets :

```
pip install scrapy

# Projet 1 : AlloCine
scrapy startproject allocine
cd allocine
scrapy genspider films www.allocine.fr

# Verifier la structure
ls -R allocine/
# allocine/spiders/films.py  items.py  pipelines.py  settings.py

# Projet 2 : Boursorama (dans un autre dossier)
cd ..
scrapy startproject boursorama
cd boursorama
scrapy genspider cac www.boursorama.com
```

Architecture Scrapy -- rappel

Engine : chef d'orchestre, distribue les requetes et collecte les reponses
Scheduler : file d'attente des URLs, dedoublonne, priorise
Downloader : fait les requetes HTTP en parallele (16 par default)
Spider : VOTRE CODE -- parse() + yield items/nouvelles requetes
Pipelines : traitement post-parse (clean, validate, store)
Cycle URL : Spider -> Engine -> Scheduler -> Downloader -> Spider.parse() -> Pipeline

TD 3.1 -- AlloCine : top 200 films

2 h 30

Cible : allocine.fr/film/meilleurs/ -- extrayez les 200 meilleurs films en suivant les liens vers les fiches detail.

Etape 1 -- Valider les selecteurs avec scrapy shell

Toujours valider ses selecteurs AVANT d'ecrire le spider. Scrapy shell met la reponse en cache tant qu'il est ouvert.

```
# Dans le terminal (dossier allocine/)
scrapy shell "https://www.allocine.fr/film/meilleurs/"

# Dans le REPL Scrapy :
>>> response.status          # doit etre 200
>>> response.css("h2.meta-title a").getall()[:3]
>>> response.css("h2.meta-title a::attr(href)").getall()[:3]
>>> response.css("a.button--right::attr(href)").get()    # lien page suivante

# Verifier une fiche detail
>>> fetch("/film/fichefilm_gen_cfilm=123456.html")
>>> response.css("h1::text").get()
>>> response.css(".stareval-note::text").get()
>>> response.css(".meta-body-direction a::text").get()
>>> exit()
```

Etape 2 -- items.py

```
# allocine/items.py
import scrapy

class FilmItem(scrapy.Item):
    titre          = scrapy.Field()
    annee          = scrapy.Field()
    realisateur    = scrapy.Field()
    note_presse    = scrapy.Field() # float
    note_spectateurs = scrapy.Field() # float
    url            = scrapy.Field()
```

Etape 3 -- Spider avec follow des liens

```

# allocine/spiders/films.py
import scrapy
from allocine.items import FilmItem

class FilmsSpider(scrapy.Spider):
    name = "films"
    start_urls = ["https://www.allocine.fr/film/meilleurs/"]
    custom_settings = {
        "DOWNLOAD_DELAY": 1.0,
        "ROBOTSTXT_OBEY": True,
    }

    def parse(self, response):
        # 1) Suivre chaque lien vers fiche detail
        for lien in response.css("h2.meta-title a::attr(href)").getall():
            yield response.follow(lien, callback=self.parse_film)

        # 2) Pagination -- lien "page suivante"
        suivant = response.css("a.button--right::attr(href)").get()
        if suivant:
            yield response.follow(suivant, callback=self.parse)

    def parse_film(self, response):
        def safe(sel):
            v = response.css(sel).get()
            return v.strip() if v else ""

        yield FilmItem(
            titre = safe("h1::text"),
            annee = safe(".meta-body-item strong::text"),
            realisateur = safe(".meta-body-direction a::text"),
            note_presse = safe(".stareval-note::text"),
            note_spectateurs = safe(".stareval-note:last-child::text"),
            url = response.url,
        )

```

Etape 4 -- pipelines.py

```

# allocine/pipelines.py
from itemadapter import ItemAdapter

class CleanPipeline:
    """Nettoie et caste les donnees."""
    def process_item(self, item, spider):
        a = ItemAdapter(item)
        for field in ["titre", "realisateur"]:
            if a.get(field):
                a[field] = a[field].strip()
        for field in ["note_presse", "note_spectateurs"]:
            try:
                raw = (a.get(field) or "").replace(",", ".")
                a[field] = float(raw)
            except (ValueError, TypeError):
                a[field] = None
        return item

```

Etape 5 -- settings.py essentiels

```
# allocine/settings.py (extrait)
BOT_NAME = "allocine"
USER_AGENT = "IPSSI-scraper (+contact@ipssi.fr)"
ROBOTSTXT_OBEY = True
DOWNLOAD_DELAY = 1.0
RANDOMIZE_DOWNLOAD_DELAY = True
CONCURRENT_REQUESTS = 4
CONCURRENT_REQUESTS_PER_DOMAIN = 2
AUTOTHROTTLING_ENABLED = True
AUTOTHROTTLING_START_DELAY = 1.0
AUTOTHROTTLING_MAX_DELAY = 10.0
RETRY_ENABLED = True
RETRY_TIMES = 3
RETRY_HTTP_CODES = [500, 502, 503, 429]
ITEM_PIPELINES = {
    "allocine.pipelines.CleanPipeline": 100,
}
FEEDS = {
    "films.json": {"format": "json", "encoding": "utf-8", "overwrite": True},
    "films.csv" : {"format": "csv", "encoding": "utf-8", "overwrite": True},
}
```

Lancer le crawl

```
# Dans le dossier allocine/
scrapy crawl films -L INFO
# -L INFO : logs propres, affiche seulement les infos importantes

# Verifier le resultat
python -c "import json; d=json.load(open('films.json')); print(len(d),'films')"
```

TD 3.2 -- Boursorama : palmares CAC 40

2 h 30

Cible : [boursorama.com/bourse/actions/palmares/france/](https://www.boursorama.com/bourse/actions/palmares/france/) -- extrayez les actions du CAC 40.

Etape 1 -- Scrapy shell sur Boursorama

```
scrapy shell "https://www.boursorama.com/bourse/actions/palmares/france/"

# Identifier les selecteurs CSS
>>> response.css("table.c-table tr").getall()[:2]
>>> response.css("td.c-table__cell a::text").getall()[:5]
>>> response.css("td.c-table__cell::text").getall()[:20]

# Le code ISIN est souvent dans l'URL du lien
>>> response.css("td.c-table__cell a::attr(href)").getall()[:3]
```

Etape 2 -- Items + SQLitePipeline

```

# boursorama/items.py
import scrapy

class ActionItem(scrapy.Item):
    libelle = scrapy.Field()
    cours = scrapy.Field() # float
    variation = scrapy.Field() # float (ex: -0.53 pour -0.53%)
    volume = scrapy.Field() # int
    isin = scrapy.Field() # cle UNIQUE en BDD


# boursorama/pipelines.py
import sqlite3
from itemadapter import ItemAdapter

DDL = '''CREATE TABLE IF NOT EXISTS actions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    libelle TEXT NOT NULL,
    cours REAL,
    variation REAL,
    volume INTEGER,
    isin TEXT UNIQUE,
    scraped_at TEXT DEFAULT CURRENT_TIMESTAMP
)'''

class SQLitePipeline:
    def open_spider(self, spider):
        self.cx = sqlite3.connect("bourse.db")
        self.cx.execute(DDL); self.cx.commit()

    def process_item(self, item, spider):
        a = ItemAdapter(item)
        try:
            self.cx.execute(
                "INSERT OR IGNORE INTO actions (libelle,cours,variation,volume,isin) "
                "VALUES (:libelle,:cours,:variation,:volume,:isin)", dict(a))
            self.cx.commit()
        except sqlite3.Error as e:
            spider.logger.error(f"SQLite: {e}")
        return item

    def close_spider(self, spider):
        n = self.cx.execute("SELECT COUNT(*) FROM actions").fetchone()[0]
        spider.logger.info(f"BDD : {n} actions en base")
        self.cx.close()

```

Etape 3 -- Spider Boursorama

```
# boursorama/spiders/cac.py
import scrapy
from boursorama.items import ActionItem

class CacSpider(scrapy.Spider):
    name = "cac"
    start_urls = ["https://www.boursorama.com/bourse/actions/palmares/france/"]

    def parse(self, response):
        for row in response.css("table.c-table tbody tr"):
            cells = row.css("td.c-table__cell")
            if len(cells) < 5: continue
            lien = cells[0].css("a")
            href = lien.attrib.get("href", "")
            # Extraire le code ISIN depuis l'URL (ex: /cours/1rXXXX/)
            isin = href.split("/")[-2] if href else ""
            try:
                cours = float(cells[1].css("::text").get("0").replace(",",".").strip())
                variation = float(cells[2].css("::text").get("0").replace(",",".").replace("%","").strip())
                volume = int(cells[3].css("::text").get("0").replace(" ","").replace(",","").strip() or 0)
            except (ValueError, TypeError):
                cours = variation = 0.0; volume = 0
            yield ActionItem(
                libelle=lien.css("::text").get("").strip(),
                cours=cours, variation=variation, volume=volume, isin=isin
            )
```

CHECKLIST AVANT RENDU

- [] scrapy crawl films => films.json + films.csv avec >= 50 films
- [] scrapy crawl cac => bourse.db avec table actions, UNIQUE(isin) fonctionne
- [] ROBOTSTXT_OBEY=True dans les deux settings.py
- [] DOWNLOAD_DELAY >= 1.0 dans les deux settings.py
- [] scrapy shell utilise pour valider les selecteurs avant le spider
- [] CleanPipeline : note castee en float, trim des textes

Pour aller plus loin -- Defis autonomes

Ces defis ancrent Scrapy dans des contextes reels que vous choisissez. Ils demandent une exploration active du web et une interpretation des donnees que vous collectez vous-meme.

Defi 1 -- Spider sur un site de votre region

Choisissez un site local que vous connaissez : journal regional, agenda culturel d'une ville, site d'une association sportive, petites annonces locales. Exigences : il doit avoir une liste d'au moins 20 elements avec titre + lien + une date ou description.

- Validez les selecteurs dans scrapy shell avant d'ecrire le spider.
- Ecrivez un spider Scrapy minimal avec un Item a 3 champs et un CleanPipeline. Exportez en CSV.
- Documentez en 5 lignes : quelles differences par rapport a AlloCine ? La structure est-elle plus ou moins previsible ? Pourquoi ?

L'interet de ce defi

Scrapy sur un site inconnu = ce que vous ferez en mission pro.

La diversite des structures HTML vous force a generaliser votre methode.

Scrapy shell vous apprend a diagnostiquer avant de coder -- reflexe essentiel.

Defi 2 -- Analyser les performances de crawl

Testez l'impact de CONCURRENT_REQUESTS sur la vitesse et la charge serveur pour AlloCine (limitez a 100 films pour eviter l'abus).

```
# Dans settings.py, tester chaque valeur et mesurer le temps
# Lancer avec : time scrapy crawl films -s CONCURRENT_REQUESTS=N -L WARNING

# Valeurs a tester : 1, 4, 8, 16
# Mesure : temps total (commande 'time') + item_scraped_count dans les stats

# Activer les stats detaillees :
STATS_DUMP = True # affiche les stats completes en fin de crawl
```

- Construisez un tableau : CONCURRENT_REQUESTS x temps x items/s.
- A partir de quelle valeur le gain devient negligeable ? Pourquoi AUTOTHROTTLER peut-il battre une valeur fixe elevee ?
- Observez item_scraped_count / response_received_count dans les stats. Que signifie un ratio < 0.5 sur AlloCine ?

Defi 3 -- SQL et interpretation financiere

Sur bourse.db apres le crawl Boursorama, ecrivez des requetes SQL qui extraient de la valeur analytique reelle.

```
-- Top 5 hausses et baisses du jour
SELECT libelle, variation, cours
FROM actions
ORDER BY variation DESC LIMIT 5;

SELECT libelle, variation, cours
FROM actions
ORDER BY variation ASC LIMIT 5;

-- Actions avec volume anormalement eleve (> 2x la mediane)
SELECT libelle, volume, cours
FROM actions
WHERE volume > (
    SELECT AVG(volume) * 2 FROM actions
)
ORDER BY volume DESC;

-- Exporter le resultat en CSV directement depuis SQLite
.mode csv
.output analyse_bourse.csv
SELECT * FROM actions ORDER BY variation DESC;
.output stdout
```

Confrontez les 5 plus grandes hausses avec les vraies nouvelles du jour (recherche web). Y a-t-il une actualite qui explique le mouvement ? Documentez 2 cas dans votre README.